Rastrear Objetos Móviles con Python

https://gemini.google.com/app/49e2f716a91ae532

User prompt: Dame un programa en Python que a partir de una secuencias de imágenes jpg me de la posición de un objeto Que se está moviendo

Response: Para rastrear la posición de un objeto en movimiento a través de una secuencia de imágenes JPG, el método más robusto y común en visión por computadora es el **análisis de fondo por sustracción** combinado con **detección de contornos** usando **OpenCV**.

Este enfoque detecta qué pixeles cambian entre fotogramas consecutivos, encuentra las regiones en movimiento y calcula el **centroide** (x, y) del objeto en cada imagen.

Requisitos previos

Asegúrate de instalar OpenCV y NumPy en tu entorno de Python:

```
pip install opencv-python numpy
```

Código en Python (`tracker.py`)

El siguiente programa lee todas las imágenes `.jpg` ordenadas de una carpeta, detecta el objeto en movimiento, calcula sus coordenadas (x, y) en cada fotograma y dibuja su trayectoria.

```
import cv2
import glob
import os
import re

def cargar_imagenes_ordenadas(ruta_carpeta):
    """Carga y ordena numéricamente las imágenes .jpg de la carpeta."""
    patron = os.path.join(ruta_carpeta, "*.jpg")
    archivos = glob.glob(patron)

    # Ordenar asegurando formato numérico natural (ej: img1, img2, img10)
    def clave_ordenacion(s):
        return [int(c) if c.isdigit() else c.lower() for c in re.split(r'(\d+)', s)]

    return sorted(archivos, key=clave_ordenacion)

def rastrear_objeto(ruta_carpeta, min_area=500):
    archivos = cargar_imagenes_ordenadas(ruta_carpeta)

    if not archivos:
        print("No se encontraron imágenes JPG en la ruta especificada.")
        return []

    # Inicializar el sustractor de fondo MOG2
    sustractor = cv2.createBackgroundSubtractorMOG2(history=500, varThreshold=50, detectShadows=False)

    posiciones = []  # Lista de tuplas: (indice, nombre_archivo, x, y)
    trayectoria = [] # Historial para dibujar el rastro del objeto

    print(f"Procesando {len(archivos)} imágenes...\n")

    for idx, ruta in enumerate(archivos):
        frame = cv2.imread(ruta)
        if frame is None:
            continue

        # 1. Aplicar un desfoque gaussiano para reducir ruido en la imagen
        blur = cv2.GaussianBlur(frame, (5, 5), 0)

        # 2. Obtener la máscara de primer plano (área en movimiento)
        fg_mask = sustractor.apply(blur)

        # 3. Operaciones morfológicas para eliminar pequeñas imperfecciones y cerrar huecos
        kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5, 5))
        fg_mask = cv2.morphologyEx(fg_mask, cv2.MORPH_OPEN, kernel)
        fg_mask = cv2.morphologyEx(fg_mask, cv2.MORPH_CLOSE, kernel)

        # 4. Encontrar los contornos de las zonas en movimiento
        contornos, _ = cv2.findContours(fg_mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)

        centroide = None
```

```
        if contornos:
            # Filtrar por área y tomar el contorno más grande (el objeto principal)
            contorno_max = max(contornos, key=cv2.contourArea)

            if cv2.contourArea(contorno_max) > min_area:
                # Calcular momentos de la imagen para hallar el centro de masa (centroide)
                M = cv2.moments(contorno_max)
                if M["m00"] != 0:
                    cx = int(M["m10"] / M["m00"])
                    cy = int(M["m01"] / M["m00"])
                    centroide = (cx, cy)
                    trayectoria.append(centroide)

                    # Dibujar bounding box y centroide en el frame
                    x, y, w, h = cv2.boundingRect(contorno_max)
                    cv2.rectangle(frame, (x, y), (x + w, y + h), (0, 255, 0), 2)
                    cv2.circle(frame, (cx, cy), 5, (0, 0, 255), -1)
                    cv2.putText(frame, f"Pos: ({cx}, {cy})", (cx + 10, cy - 10),
                                cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 255, 0), 2)

        # Dibujar la línea de la trayectoria histórica
        for i in range(1, len(trayectoria)):
            if trayectoria[i - 1] and trayectoria[i]:
                cv2.line(frame, trayectoria[i - 1], trayectoria[i], (255, 0, 0), 2)

        nombre_foto = os.path.basename(ruta)
        posiciones.append((idx, nombre_foto, centroide))

        # Mostrar coordenadas en la consola
        coord_str = f"X: {centroide[0]}, Y: {centroide[1]}" if centroide else "Objeto no detectado / Estático"
        print(f"Frame [{idx:03d}] - {nombre_foto}: {coord_str}")

        # Visualización en vivo
        cv2.imshow("Tracking de Objeto", frame)
        cv2.imshow("Mascara de Movimiento", fg_mask)

        # Presionar 'q' o ESC para salir
        key = cv2.waitKey(100) & 0xFF  # 100 ms de delay por frame (ajustable)
        if key == 27 or key == ord('q'):
            break

    cv2.destroyAllWindows()
    return posiciones

# ========================================
# Ejemplo de Ejecución
# ========================================
if __name__ == "__main__":
    # Cambia esto por la ruta de tu carpeta con imágenes .jpg
    CARPETA_IMAGENES = "./secuencia_jpg"

    # min_area: ignora objetos detectados menores a este tamaño en píxeles
    resultados = rastrear_objeto(CARPETA_IMAGENES, min_area=300)
```

¿Cómo funciona la detección?

1. **`BackgroundSubtractorMOG2`**: Mantiene un modelo estadístico de la escena para aprender cuál es el fondo estático e identificar la región que cambia de posición.
2. **Filtrado Morfológico (`MORPH_OPEN` / `MORPH_CLOSE`)**: Limpia el ruido producido por cambios ligeros de iluminación o sombras desvanecidas.
3. **Cálculo de Momentos (`cv2.moments`)**: Obtiene el centroide físico del área detectada a partir de la fórmula de momentos espaciales:

$$cx = \frac{M_{10}}{M_{00}}, \quad cy = \frac{M_{01}}{M_{00}}$$

Alternativas según las características de tus imágenes

- **Si la cámara también se mueve (movimiento de cámara):** La sustracción de fondo no funcionará directamente. En ese caso, conviene usar seguimiento por flujo óptico (**Farneback / Lucas-Kanade**) o usar algoritmos de seguimiento basados en reidentificación tipo **CSRT** (`cv2.TrackerCSRT_create()`) inicializando el área en el primer frame.

- **Si conoces el color del objeto:** Puedes sustituir la sustracción de fondo por una segmentación en el espacio de color **HSV** usando `cv2.inRange()`.
